

Sensor Systems

Laboratory 02
Programming STM32F4 MCU

19.09.2026 / 21.09.2026 | Federica Villa, Arianna De Vecchi

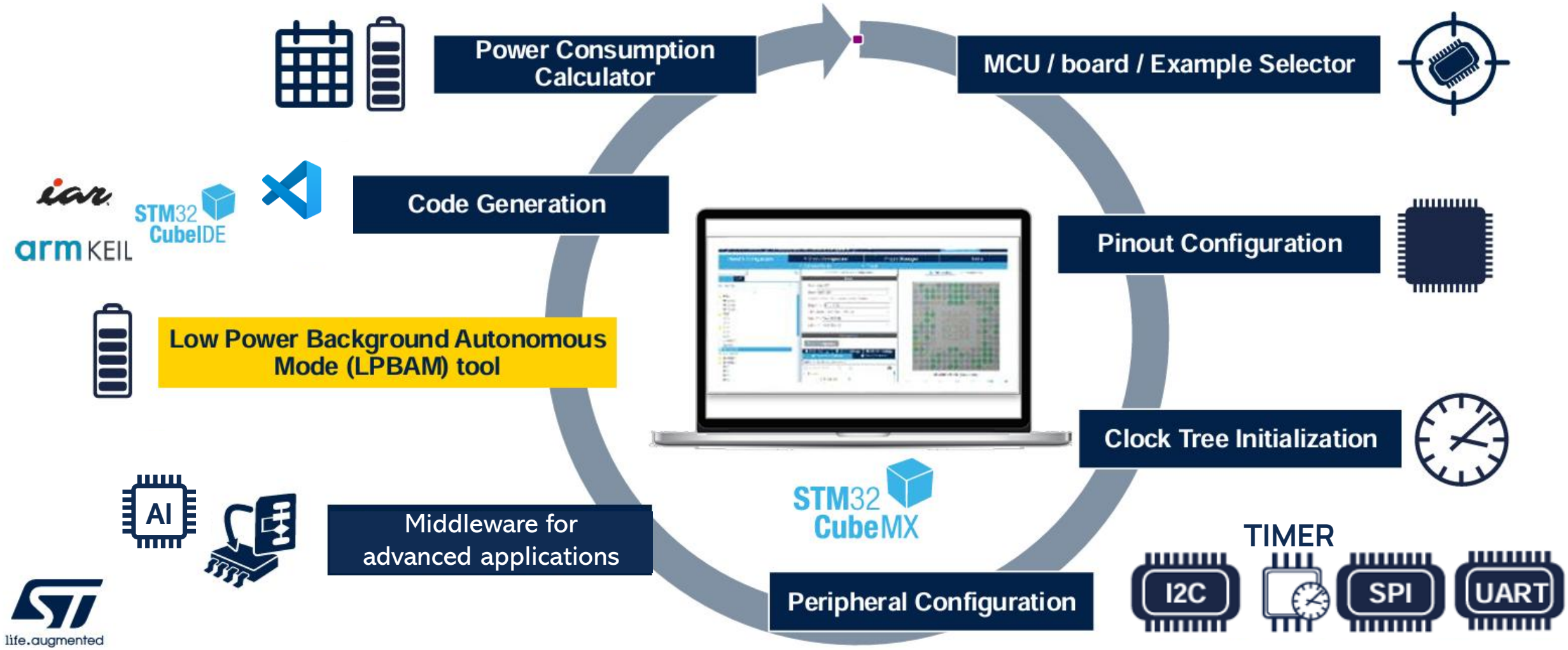
Contents

1. STM32CubeMX Basics
2. Hardware & Pinout Mapping
3. Code Generation
4. Debugger

1. STM32CubeMX Basics

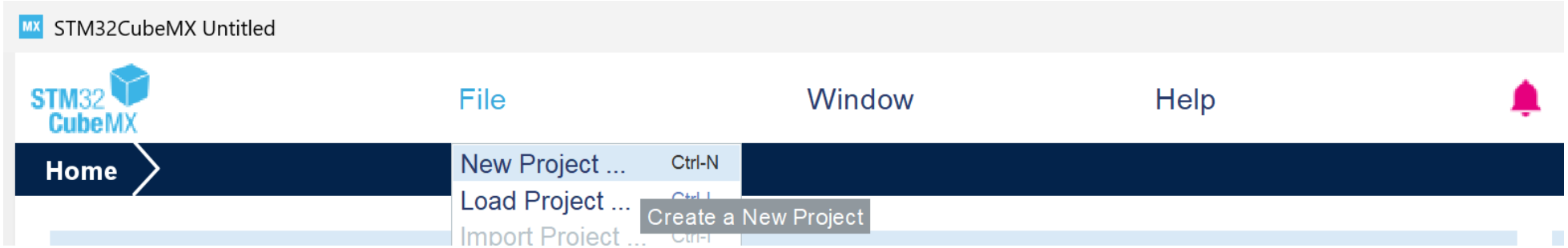
01

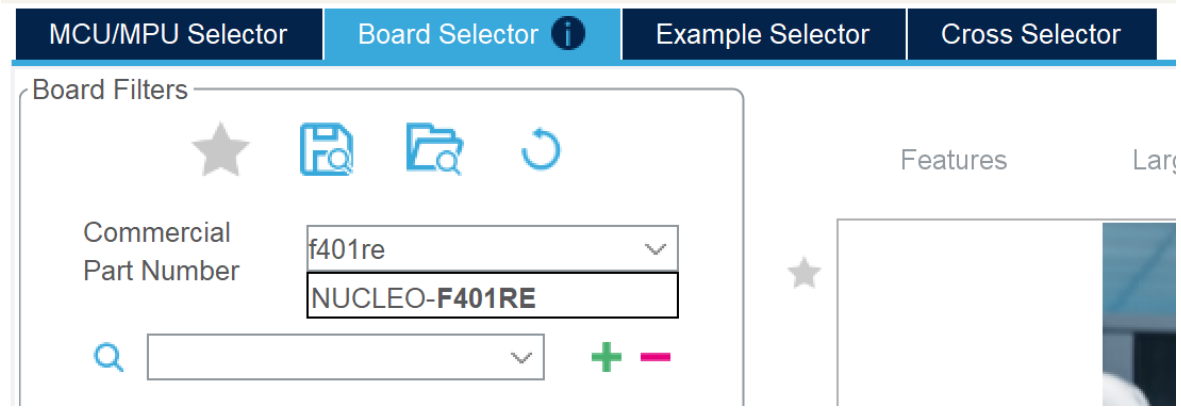
The STM32 CubeMX Workflow



MCU/Board Selection

File › New Project …

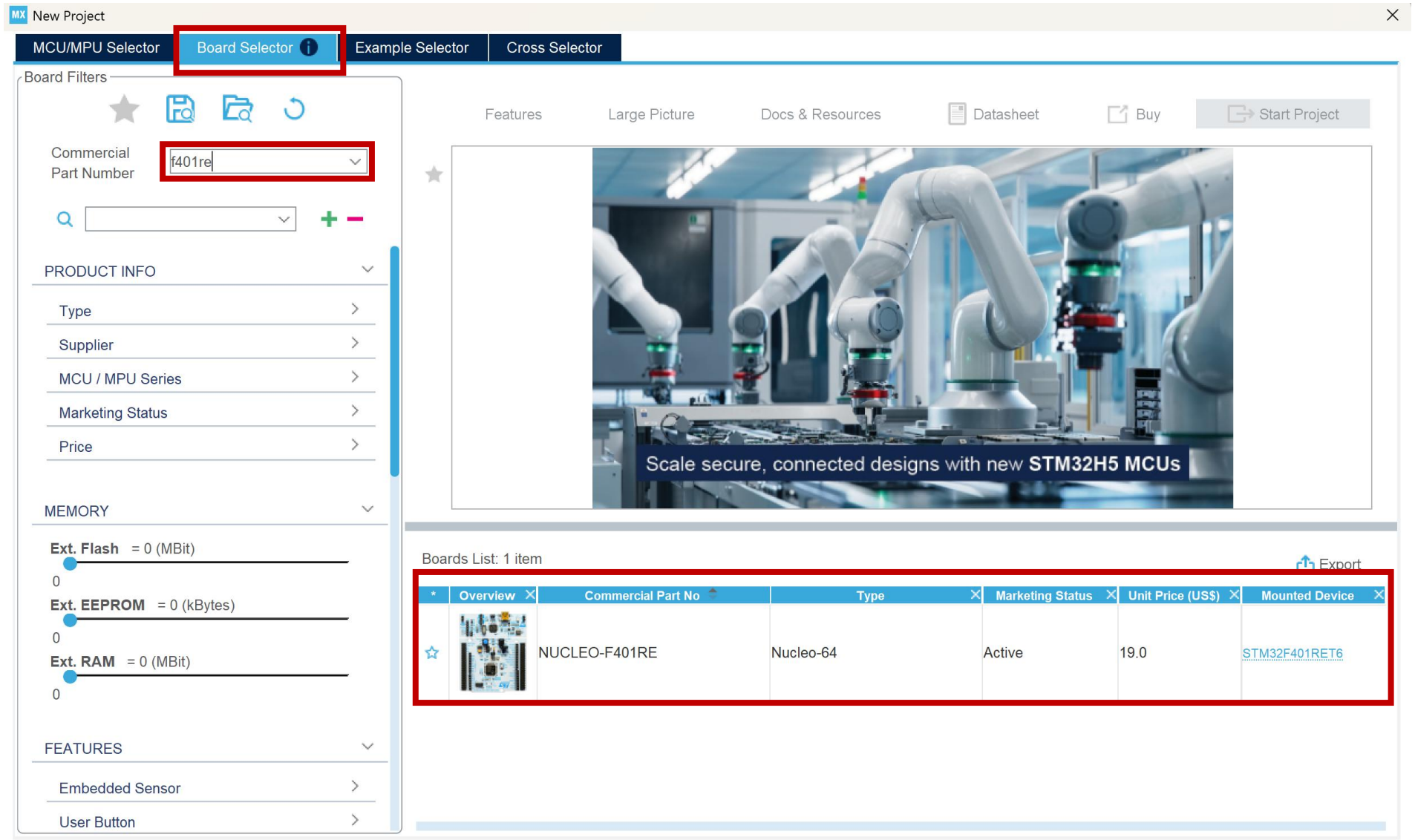




MCU Selector: Pick the exact bare microcontroller to start a custom project.

Board Selector: Pick a commercial development board (e.g., Nucleo) to automatically load its default pin configurations.

MCU/Board Selection



MCU/Board Selection

Features | Large Picture | Docs & Resources | Datasheet | Buy | Start Project

STM32F4 Series

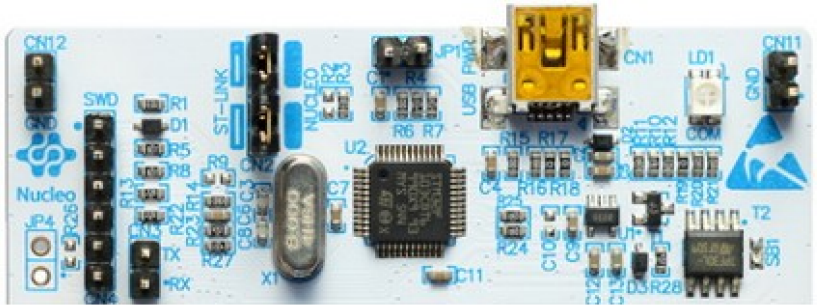
NUCLEO-F401RE — **STM32 Nucleo-64 development board with STM32F401RE MCU, supports Arduino and ST morpho connectivity**

ACTIVE
Product is in mass production

Part Number : NUCLEO-F401RE
Commercial Part Number : NUCLEO-F401RE

Unit Price (US$) : **19.0**

Mounted Device : STM32F401RET6



The STM32 Nucleo-64 board provides an affordable and flexible way for users to try out new concepts and build prototypes by choosing from the various combinations of performance and power consumption features provided by the STM32 microcontroller. For the compatible boards, the internal or external SMPS significantly reduces power consumption in Run mode.
The ARDUINO® Uno V3 connectivity support and

Boards List: 1 item

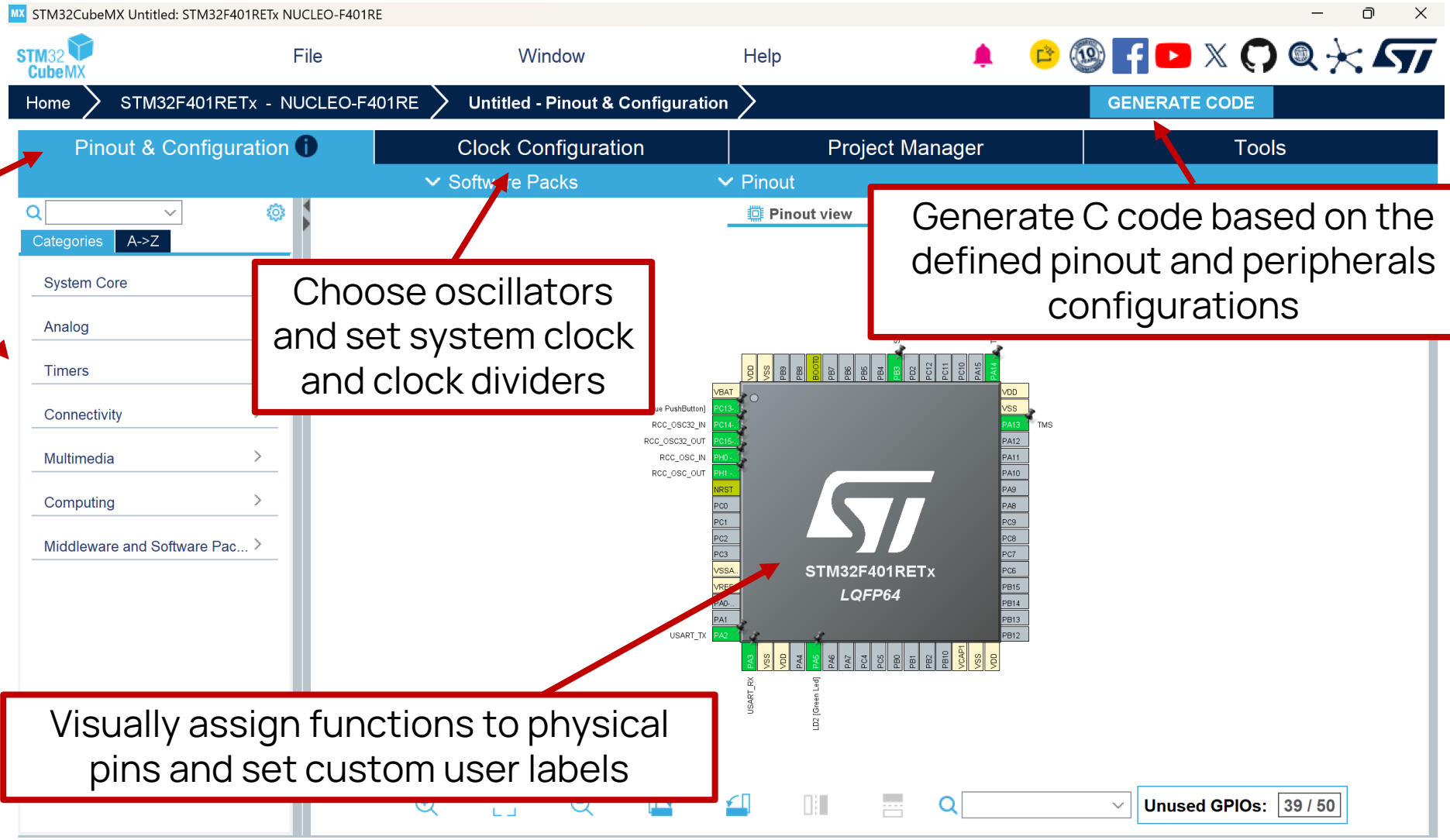
Export

*	Overview	Commercial Part No	Type	Marketing Status	Unit Price (US$)	Mounted Device
☆		NUCLEO-F401RE	Nucleo-64	Active	19.0	STM32F401RET6

When asked if peripherals should be initialized in default mode, say “Yes” .

STM32 CubeMX User Interface

Choose peripherals to use and configure them according to our application



Choose oscillators and set system clock and clock dividers

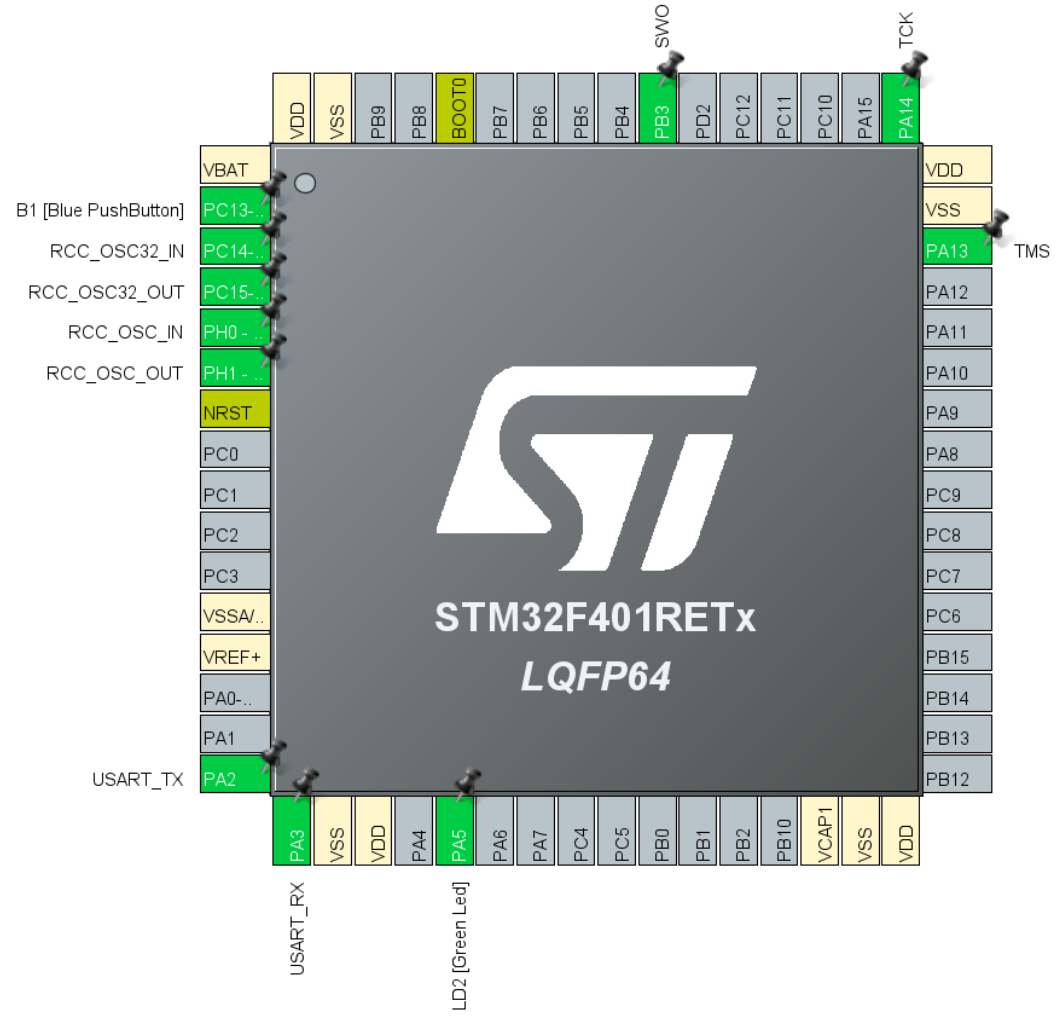
Generate C code based on the defined pinout and peripherals configurations

Visually assign functions to physical pins and set custom user labels

2. Hardware & Pinout Mapping

02

Pin assignment



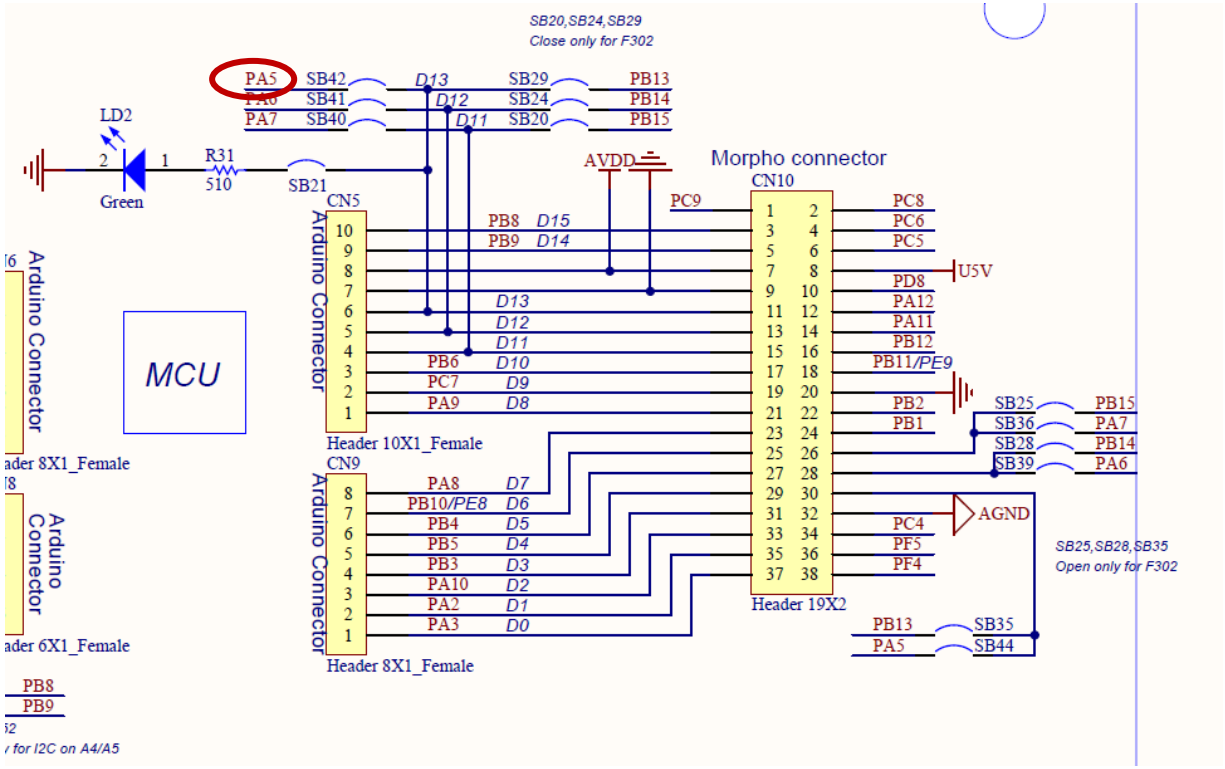
The CubeMX interface already maps the pins for the selected NUCLEO board.

However, we will need to configure the pins for the POLIMI board ourselves! This information is in the schematics presented in the previous lab.

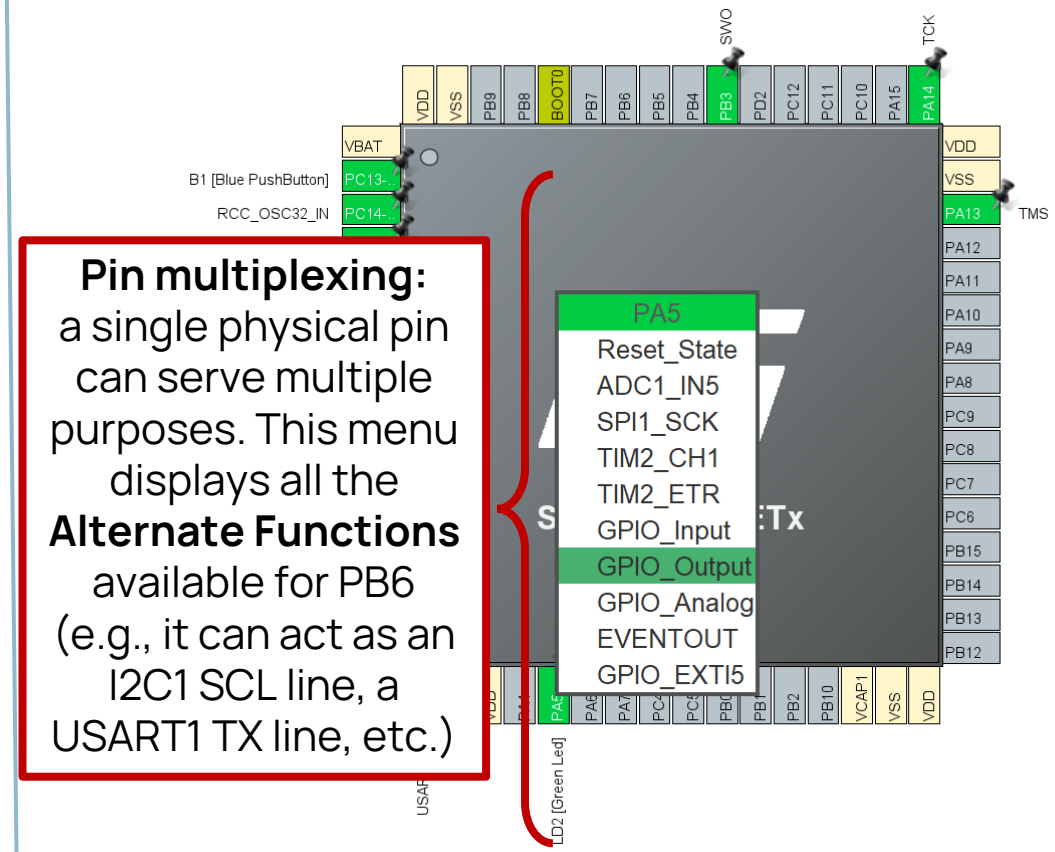
Pin assignment

Example: driving the Green LED on the NUCLEO

Schematic: Pin PA5 is physically connected to the Green LED.



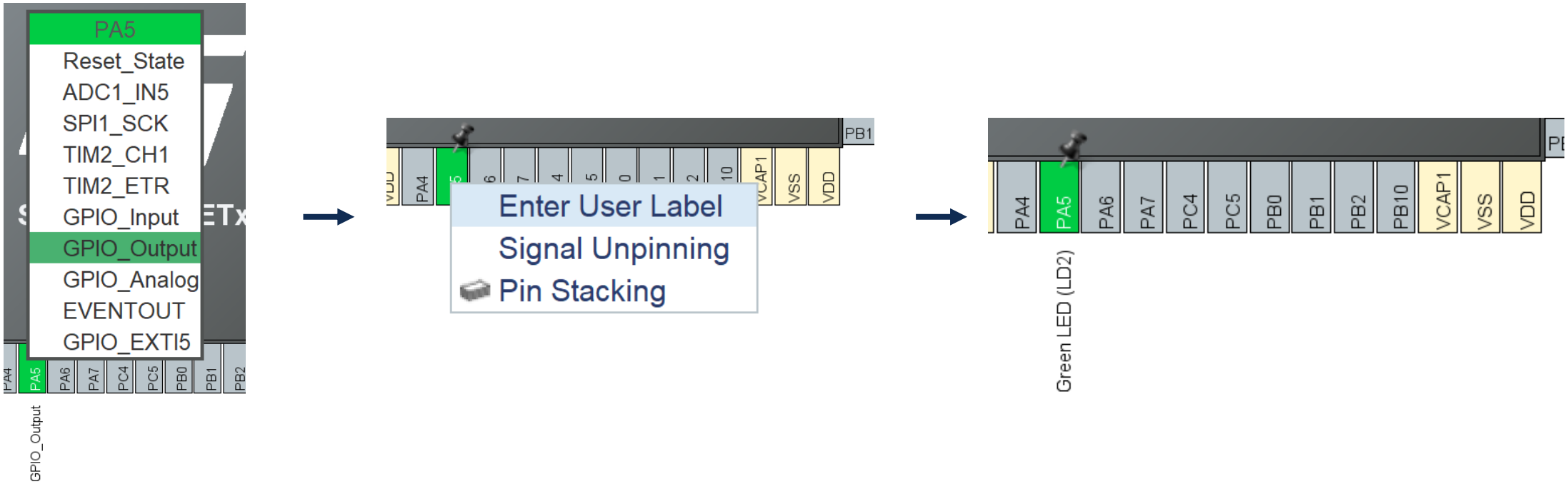
CubeMX Action: Left-click on PA5, it is already set as **GPIO_Output**.



Pin multiplexing: a single physical pin can serve multiple purposes. This menu displays all the **Alternate Functions** available for PB6 (e.g., it can act as an I2C1 SCL line, a USART1 TX line, etc.)

Pin assignment

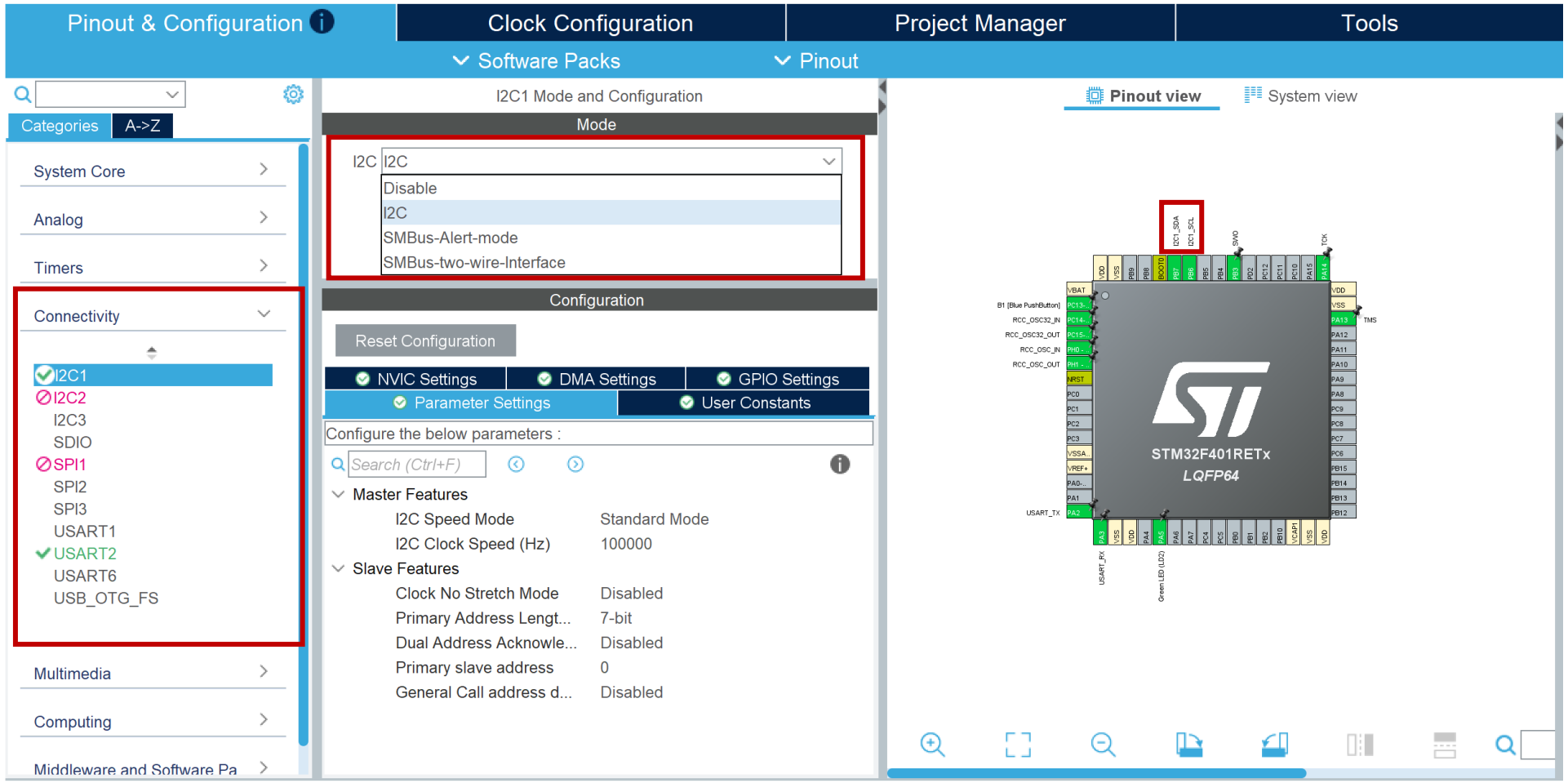
Best Practice: Right-click on a pin to set a label to the pin and name it to match the schematic.



Pin assignment

Example: Activating a peripheral (e.g., I2C)

CubeMX Action: Expand the "Connectivity" menu on the left and enable **I2C1** (Set I2C Mode).



CubeMX **automatically** assigns and highlights the ***default*** hardware pins for I2C1 (SCL and SDA, which in this case are assigned to PB6 and PB7)

Pin assignment

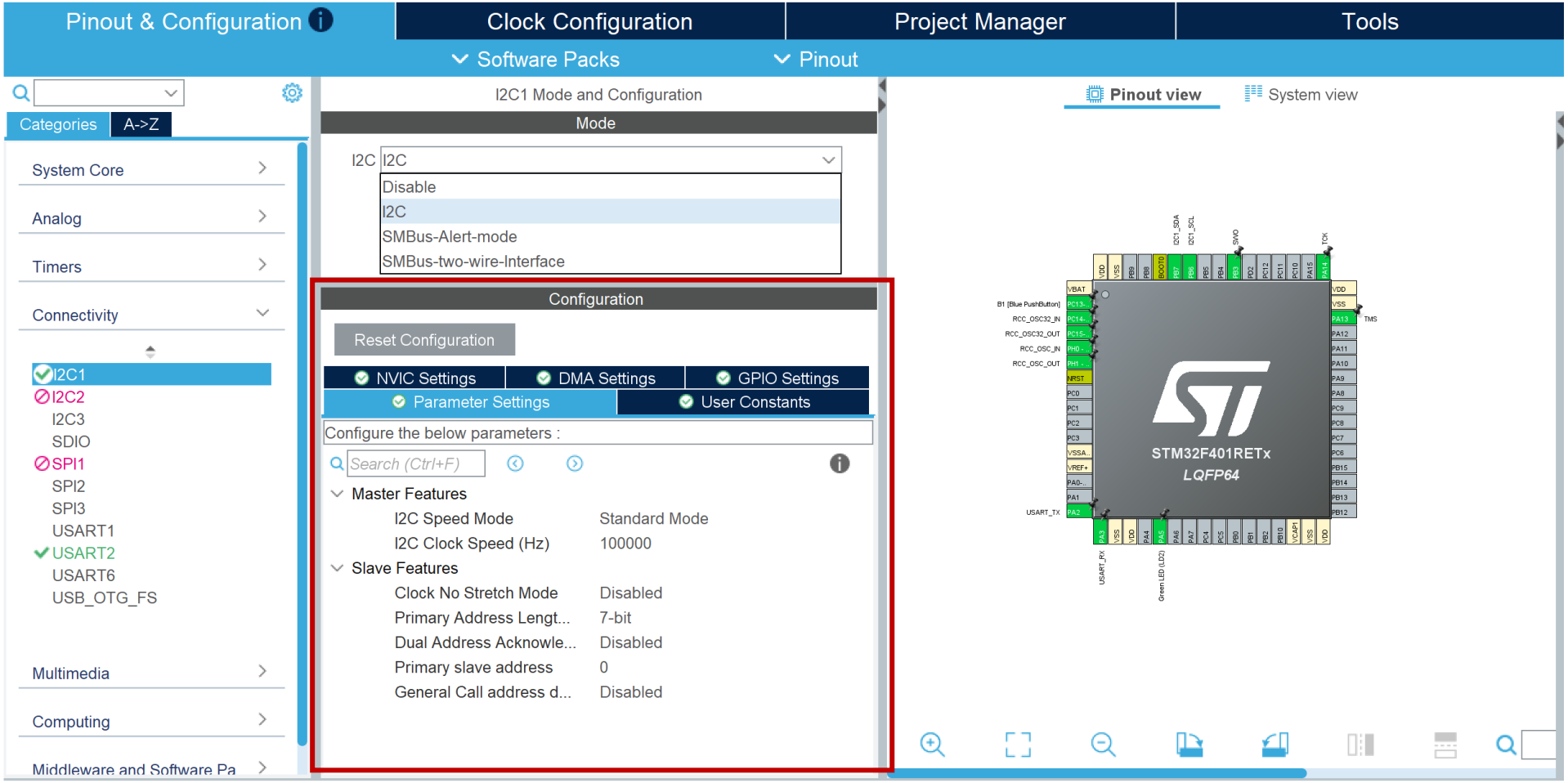


Reality Check: Do these default pins match our schematic?

If our PCB designer used different pins with respect to the default ones, we must **remap** them in CubeMX to match the ones on the physical board.

You *can* assign the correct pins directly by clicking them, according to your schematic. However, they will turn **orange**. This is a warning: the pin is reserved, but the actual I2C peripheral is still disabled in the menu!

Pin assignment



CubeMX allows us to configure the peripheral as needed: we will see that each peripheral has different configuration options.

Try it yourselves!

Create a new project and try to look at the various pins and functionalities, so that you get familiar with the interface.

In the meantime, we will distribute the boards that will be needed for the next steps.

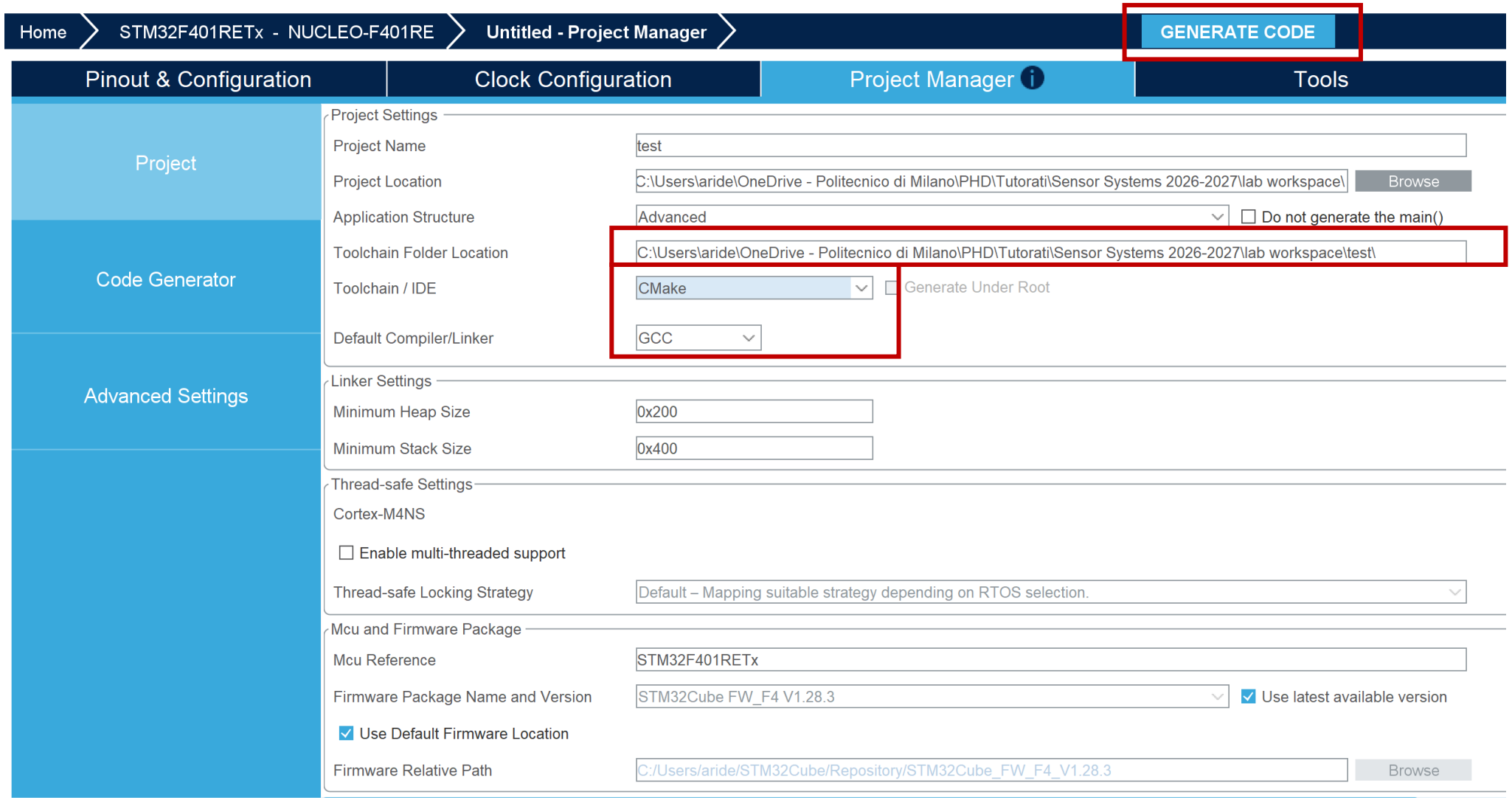
If you had any issues with the set-up, please let us know.

3. Code Generation

03

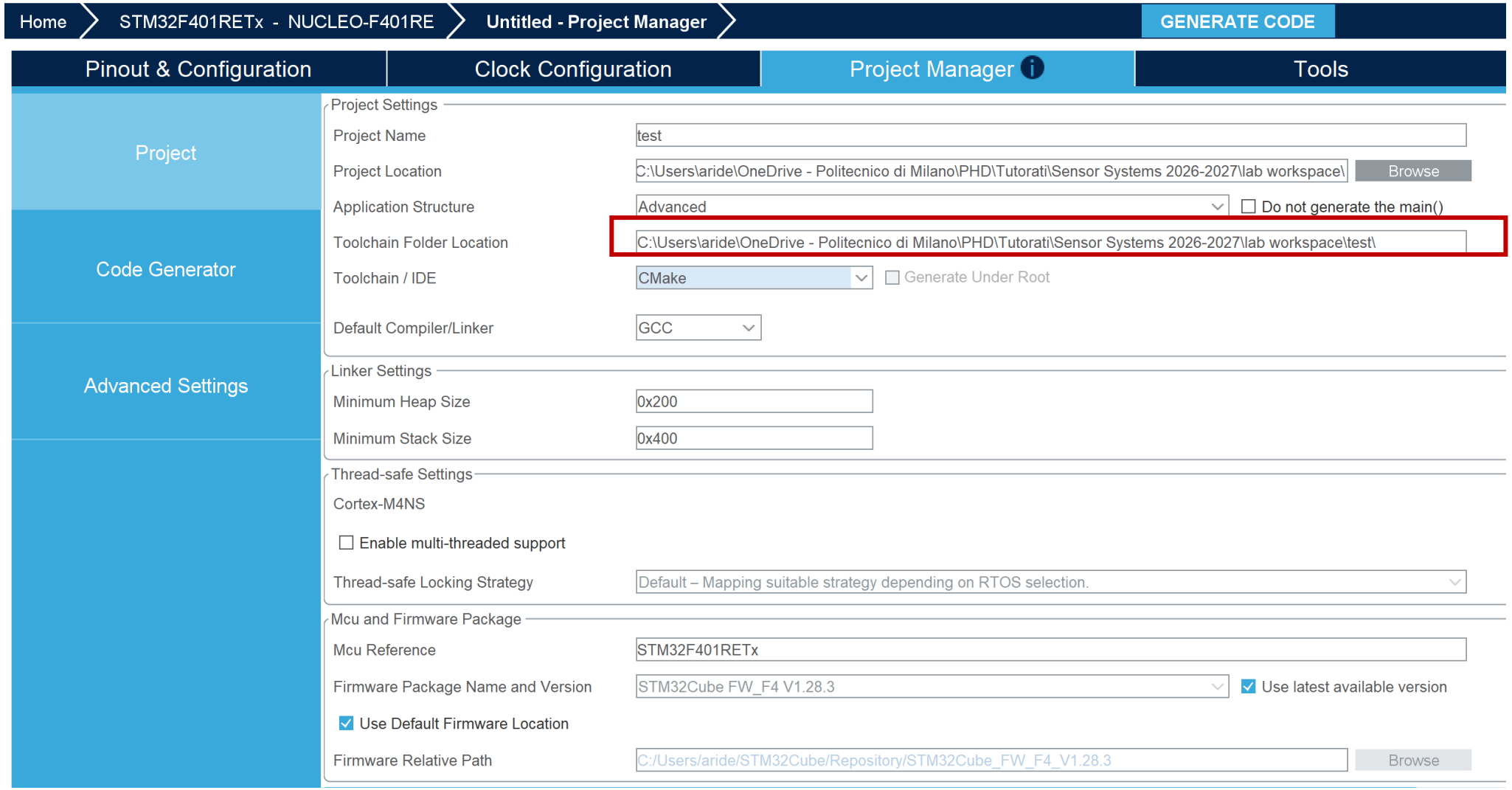
Code generation

1. Define project name and project location on your PC.
2. Then select Cmake as Toolchain/IDE
3. Click on “Generate Code”



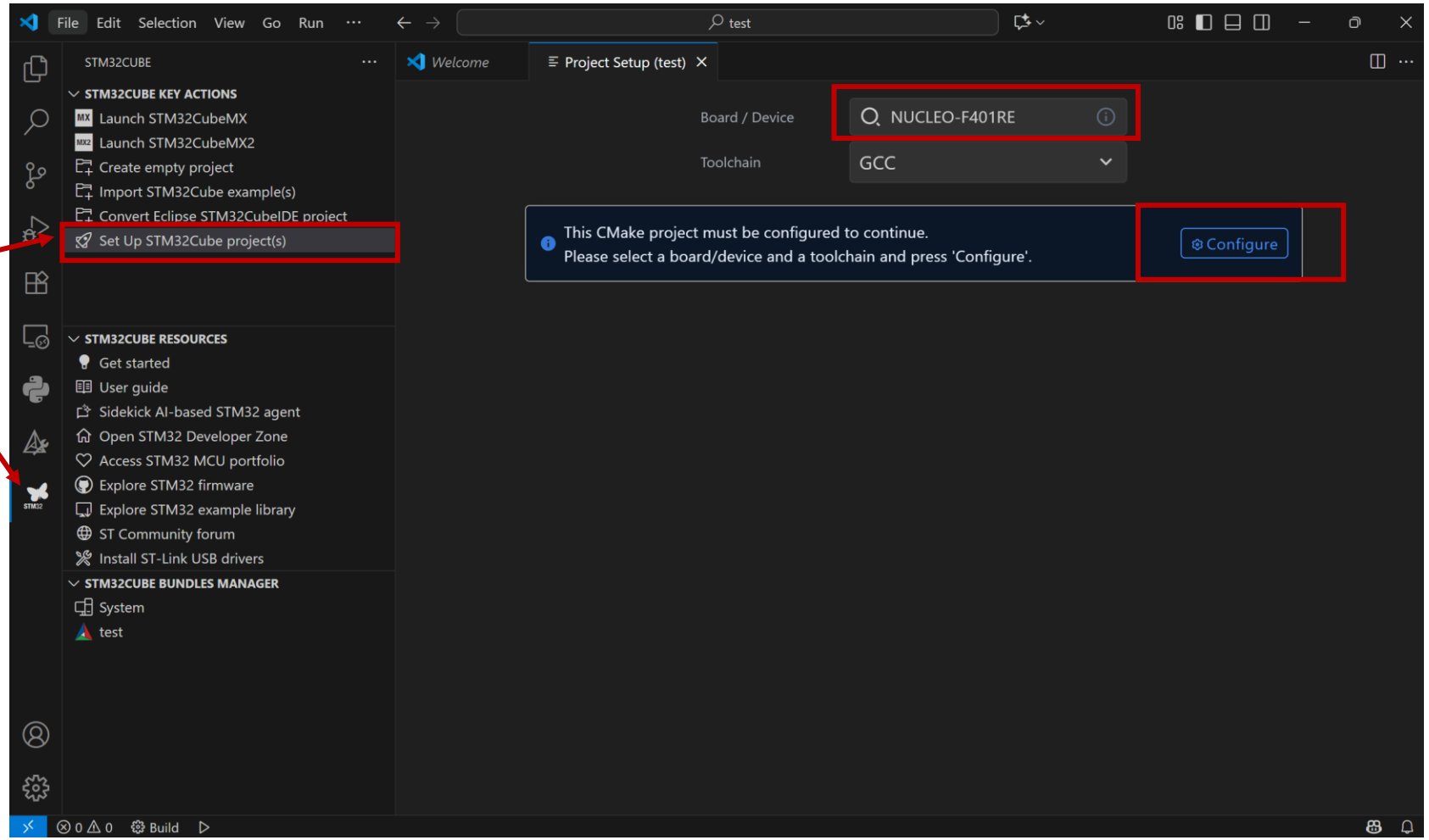
Code generation

Tip: save all your projects in a defined folder!



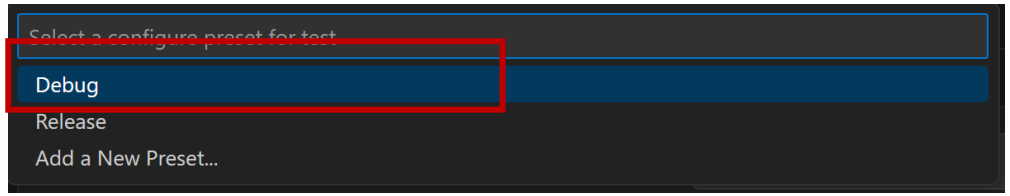
Code generation

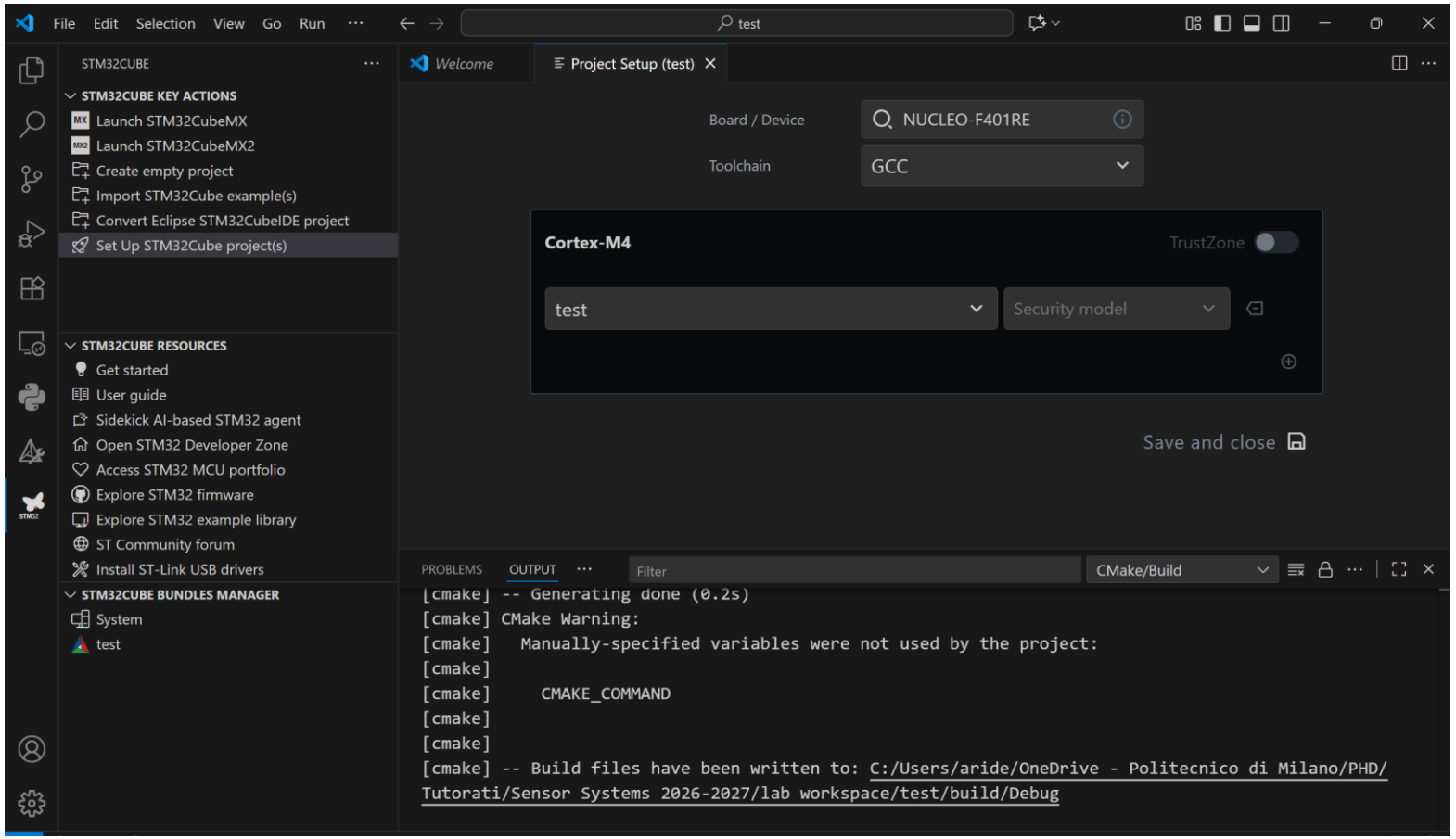
Open the project folder on Visual Studio Code (VSC) and click on the STM32 Cube extension, then “Set Up STM32Cube project(s)”



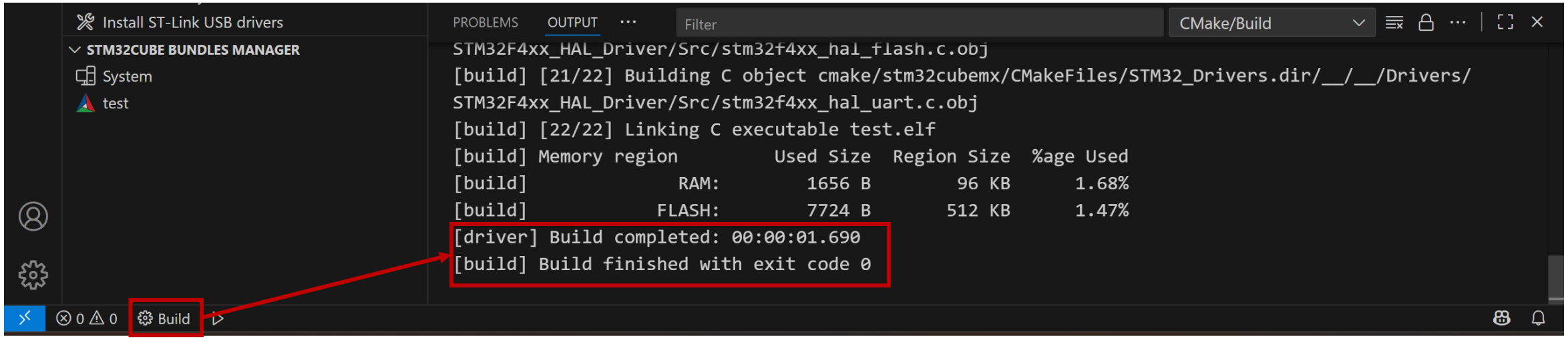
Check the Board name and click on “configure”

Code generation

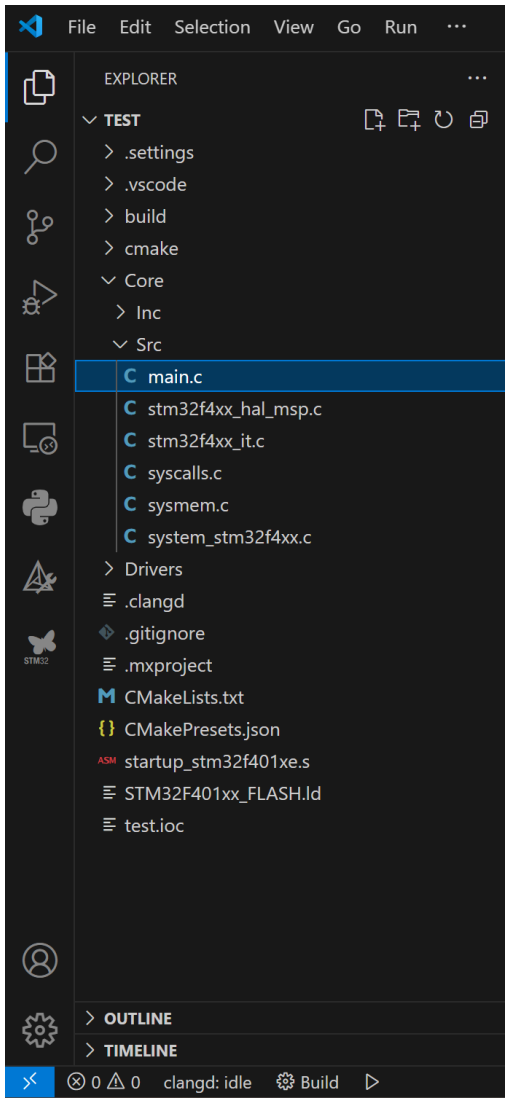




Code generation



Code generation



In the file explorer, you can see the files of the project:

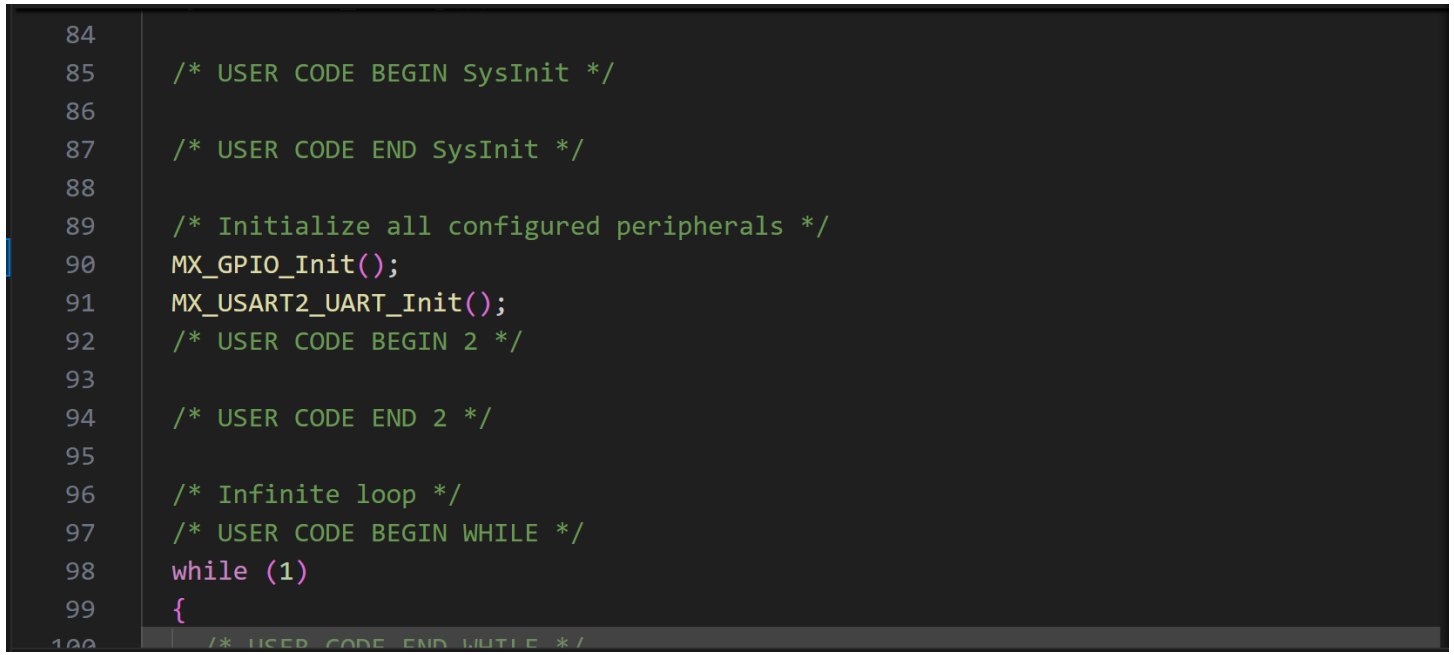
- main.c is the file we will mostly edit
- Project_name.ioc is the file that CubeMX uses for the GUI
- In Inc you will find .h files, containing configurations, etc

The main.c file is configured as we discussed in the first class:

```
int main(void)
{
        while (1)
        {
        }
}
```

Code generation

CubeMX translates our GUI settings into C functions automatically placed in main.c

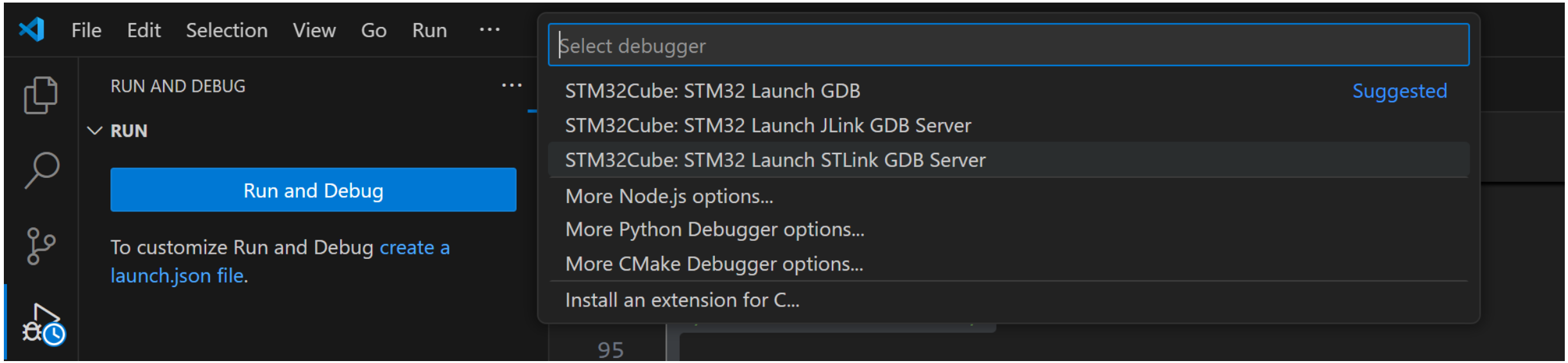


```
84
85  /* USER CODE BEGIN SysInit */
86
87  /* USER CODE END SysInit */
88
89  /* Initialize all configured peripherals */
90  MX_GPIO_Init();
91  MX_USART2_UART_Init();
92  /* USER CODE BEGIN 2 */
93
94  /* USER CODE END 2 */
95
96  /* Infinite loop */
97  /* USER CODE BEGIN WHILE */
98  while (1)
99  {
```

Always write your custom code **ONLY** between the `/* USER CODE BEGIN */` and `/* USER CODE END */` comments!

If you write outside these blocks, CubeMX will permanently delete your code upon the next generation.

Code generation



After clicking on "Run and Debug", choose "**STM32Cube: STM32 Launch STLink GDB Server**."

You will need the board to be connected to run and to debug the code.

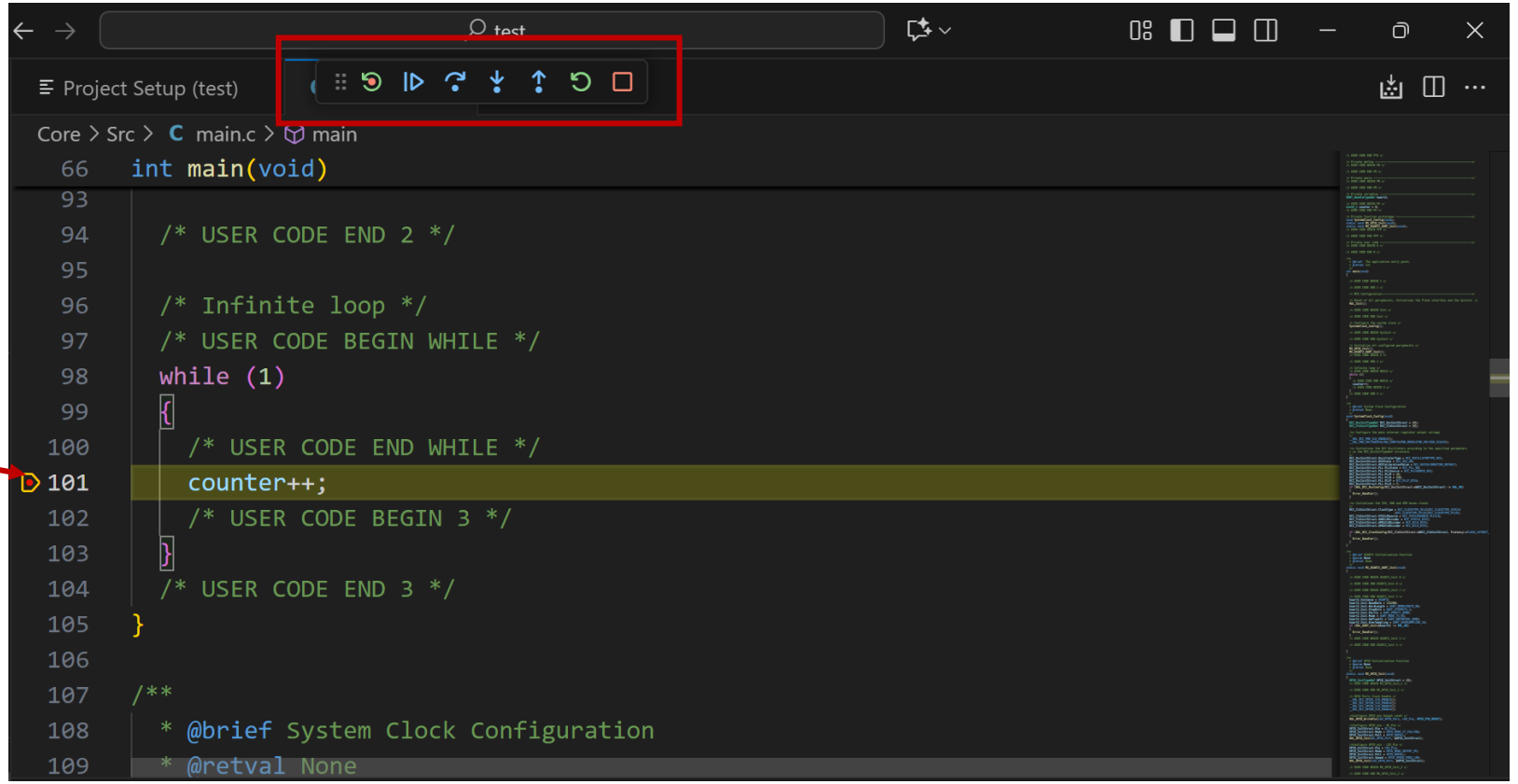
4. Debugger

04

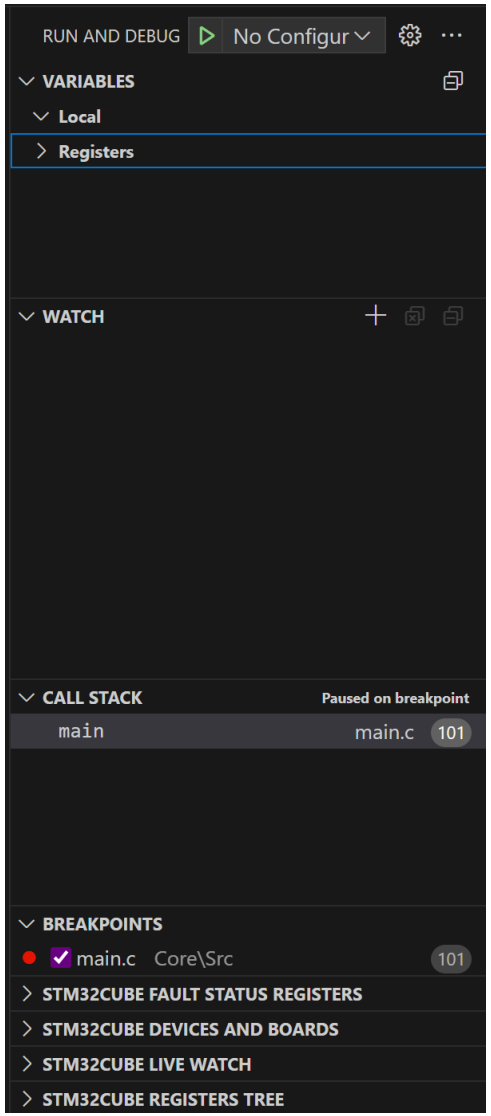
Code debug

Clicking on the dot next to a line inserts a breakpoint.
The code will stop whenever that line is reached: we can then analyze variables, registers, etc to verify our code is working correctly.
To proceed with code execution (or to stop it if something is wrong) we can use the dedicated commands.

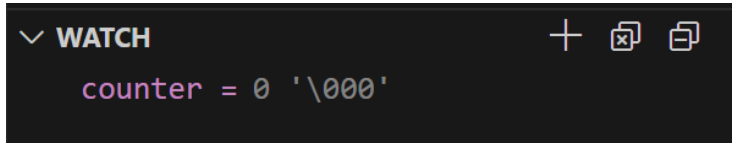
Breakpoint



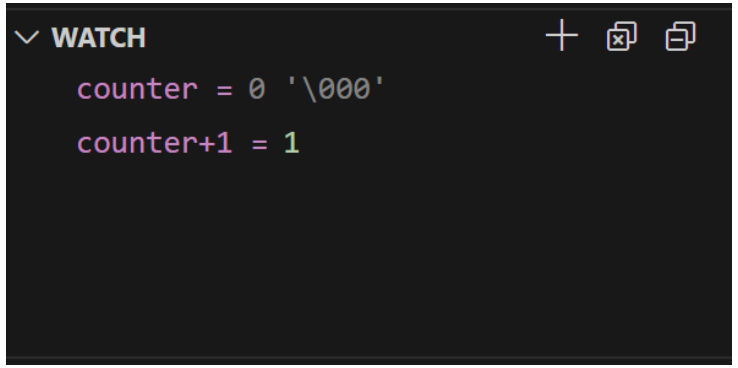
Code debug



Here we can monitor registers and variables. To check a variable's value, we might need to add it to the «watch» section.

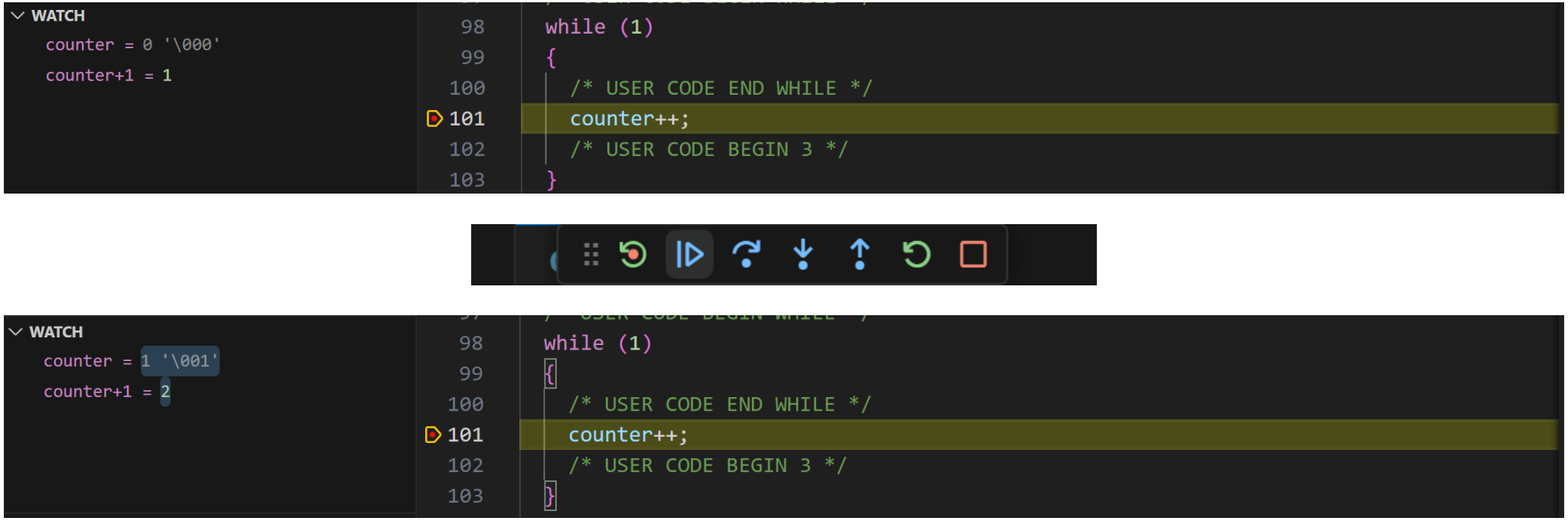


We can also add «expressions», to verify if a computation we perform in our code is correct.



Code debug

Example: counter++ increases each time we play the code, which stops at each while cycle execution.



When we are done with the debugging phase, we can remove the breakpoints and the code will run smoothly.



POLITECNICO MILANO 1863 | DEPARTMENT OF ELECTRONICS INFORMATION AND BIOENGINEERING

Arianna.devecchi@polimi.it
Federica.villa@polimi.it
www.polimi.it

POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA